QuillAudits

AUDIT REPORT

July 2026

For

TAJIR CHAIN

Table of Content

Executive Summary

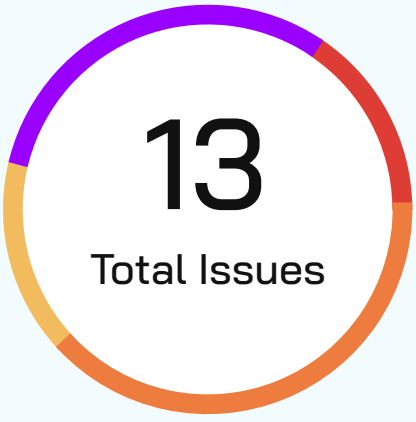
Project Name	Tajir Chain - Audit Report
Protocol Type	Vesting, ERC20
Project URL	https://www.tajirchain.com/
Overview	Tajir Chain's token and vesting layer. TajirToken is a fixed-supply, UUPS-upgradeable ERC-20 with EIP-2612 permit, minted once at initialization and locked on Ethereum L1 to back TJR as the native gas asset on the Tajir rollup. TajirTokenProxy is a thin ERC-1967 proxy wrapper deployed in front of it. TajirVestingWallet distributes the 650M TJR tokenomics allocation across eight categories, extending OpenZeppelin's VestingWalletCliff with tranche-based release nothing unlocks until the cliff, after which the allocation vests in N equal installments and adds a one-shot admin clawback of the still-unvested surplus.
Audit Scope	The scope of this Audit was to analyze the Tajir Chain Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/Tajir-Chain/tajir-chain-contracts-external
Branch	foundry
Contracts in Scope	TajirVestingWallet.sol TajirToken.sol TajirTokenProxy.sol
Commit Hash	01802d8370699dbe09ec1113df59c6388c32eb51
Language	Solidity
Blockchain	Tajir Chain , EVM
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	21st July, 2026

Updated Code Received	24th July, 2026
Review 2	27th July, 2026
Fixed In	f70f18049eeb85102bbacf947942912cacec107f
Review 3	On 27th August 2026 , The Tajir team has made changes to the token contract post-audit and requested a quick review. The changes are related to the addition of small burn mechanisms. QuillAudits Team reviewed changed and its looks good, no new vulnerabilities in changes. https://github.com/Tajir-Chain/tajir-chain-contracts-external/commit/c6c860315e7de8b9e0adb6cb13def170b4630bfd

Verify the Authenticity of Report on QuillAudits Leaderboard:

https://www.quillaudits.com/leaderboard

Number of Issues per Severity



Critical	0 (0%)
High	2 (15.38%)
Medium	5 (38.48%)
Low	2 (15.38%)
Informational	4 (30.76%)

Severity

Issues	Critical	High	Medium	Low	Informational
Open	0	0	0	0	0
Acknowledged	0	**1**	**4**	**2**	**1**
Partially Resolved	0	0	0	0	0
Resolved	0	**1**	**1**	0	**3**

Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Unprotected inherited initializer allows seizure of an uninitialized vesting wallet	High	Resolved
2	A single global revoked flag voids the vesting schedule for every asset held	High	Acknowledged
3	renounceOwnership() and transferOwnership() are inherited un-overridden	Medium	Acknowledged
4	Unguarded subtraction underflows against balance-reducing tokens	Medium	Resolved
5	initialize() never validates startTimestamp	Medium	Acknowledged
6	Clawback commits an irreversible flag with no balance-delta assertion	Medium	Acknowledged
7	Schedule parameters are read live with no clamp	Medium	Acknowledged
8	Tranche quantization lets a revocation strip a nearly-elapsed tranche	Low	Acknowledged
9	TajirToken.initialize silently rescales initialSupply	Low	Acknowledged

Issue No.	Issue Title	Severity	Status
10	revoked has no getter and its guard precedes the authorization gate	Informational	Resolved
11	withdrawUnreleased re-derives releasable() inline	Informational	Resolved
12	Floor division routes rounding remainders away from the beneficiary	Informational	Resolved
13	upgradeToAndCall inherited payable with no ETH-handling path	Informational	Acknowledged

Checked Vulnerabilities

- Access Management
- Arbitrary write to storage
- Centralization of control
- Ether theft
- Improper or missing events
- Logical issues and flaws
- Arithmetic Computations Correctness
- Race conditions/front running
- SWC Registry
- Re-entrancy
- Timestamp Dependence
- Gas Limit and Loops
- Exception Disorder
- Gasless Send
- Use of tx.origin
- Malicious libraries
- Compiler version not fixed
- Address hardcoded
- Divide before multiply
- Integer overflow/underflow
- ERC's conformance
- Dangerous strict equalities
- Tautology or contradiction
- Return values of low-level calls

- Missing Zero Address Validation
- Private modifier
- Revert/require functions
- Multiple Sends
- Using suicide
- Using delegatecall
- Upgradeable safety
- Using throw
- Using inline assembly
- Style guide violation
- Unsafe type inference
- Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

Impact

Likelihood	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- High - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- Medium - only a conditionally incentivized attack vector, but still relatively likely.
- Low - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

High Severity Issues

Unprotected inherited initializer allows seizure of an uninitialized vesting wallet

Resolved

Path

src/TajirVestingWallet.sol

Function Name

`initialize(address,uint64,uint64) (inherited), initialize(...) (8-arg, L82-111)`

Description

TajirVestingWallet inherits VestingWalletUpgradeable.initialize(address,uint64,uint64), which is declared public virtual initializer at VestingWalletUpgradeable.sol:57. The contract declares its own 8-argument initialize(...) at with a different function selector, and does not mark it override. Both functions therefore remain externally callable on the deployed contract. This was confirmed against the compiled artifact:

```
$ forge inspect src/TajirVestingWallet.sol:TajirVestingWallet methods
| initialize(address,uint64,uint64)                                | 989a8366 |
| initialize(address,uint64,uint64,uint64,uint64,uint64,address,address) | ad562987 |
```

Both paths consume the same one-shot initializer flag, so whichever executes first wins. _disableInitializers() in the constructor protects only the implementation contract, it never touches the proxy's storage.

The window is reachable because deployment is not atomic. script/DeployVesting.s.sol constructs the proxy with empty init calldata and initializes in a separate call at L172:

```
1  TajirVestingWallet impl = new TajirVestingWallet();
2  ERC1967Proxy proxy = new ERC1967Proxy(address(impl), "");   // L169  empty init data
3  wallet = TajirVestingWallet(payable(address(proxy)));
4  wallet.initialize(                                          // L172  separate transaction
5      cfg.beneficiary, startTimestamp, cfg.duration, cfg.cliff,
6      cfg.trancheDuration, cfg.totalTranches, admin, upgrader
7  );
```

Impact

Medium

An attacker who lands the 3-argument path first becomes owner() the beneficiary who receives every release() and consumes the initializer flag. The deployer's 8-argument call then reverts with InvalidInitialization, failing the deployment script.

Because DeployVesting.s.sol performs no funding (there is no transfer or mint anywhere in the script), the attacker seizes an empty wallet. The realised harm is therefore a repeatable denial of service on deployment plus wasted gas, rather than direct theft. Every redeployment attempt is equally front-runnable, so an attacker can block vesting deployment indefinitely.

Direct fund loss requires a compounding operational error funding a wallet whose initialization reverted. Should that occur, the seized wallet is unrecoverable: admin and upgrader are address(0) (no clawback, no upgrade path) and trancheDuration == 0 makes _vestingSchedule divide by zero at, bricking every value-exit path.

Likelihood

High

Permissionless, requires only mempool observation, and no capital.

POC

```
function test_shadowInitializerSeizure() public {
    TajirVestingWallet impl = new TajirVestingWallet();
    ERC1967Proxy proxy = new ERC1967Proxy(address(impl), "");   // uninitialized
    TajirVestingWallet w = TajirVestingWallet(payable(address(proxy)));

    // attacker front-runs with the inherited 3-arg selector
    vm.prank(attacker);
    VestingWalletUpgradeable(payable(address(w)))
        .initialize(attacker, uint64(block.timestamp), 365 days);

    assertEq(w.owner(), attacker);              // attacker is now beneficiary
    assertEq(w.trancheDuration(), 0);           // schedule params never set

    // the legitimate deployment call now reverts
    vm.expectRevert();                          // InvalidInitialization
    w.initialize(beneficiary, uint64(block.timestamp), 365 days,
                 30 days, 30 days, 11, admin, upgrader);
}
```

Recommendation

Apply both fixes

1. Override and permanently disable the inherited initializer:

```
function initialize(address, uint64, uint64) public pure override {
    revert("TajirVestingWallet: use the 8-argument initialize");
}
```

2. Make deployment atomic by passing the initialization calldata as the proxy constructor's data argument, removing the window entirely:

```
bytes memory initData = abi.encodeCall(
    TajirVestingWallet.initialize,
    (cfg.beneficiary, startTimestamp, cfg.duration, cfg.cliff,
     cfg.trancheDuration, cfg.totalTranches, admin, upgrader)
);
ERC1967Proxy proxy = new ERC1967Proxy(address(impl), initData);
```

High Severity Issues

A single global revoked flag voids the vesting schedule for every asset held

Acknowledged

Path

src/TajirVestingWallet.sol

Function Name

`withdrawUnreleased(address,address payable) _vestingSchedule(uint256,uint64)`

Description

The vesting wallet stores revocation as a single global boolean (bool revoked, L35) while allocations are derived per asset from live balances.

withdrawUnreleased(token, to) accepts an admin-chosen token address and writes that one flag at

```
function withdrawUnreleased(address token, address payable to) external {
    TajirVestingWalletStorage storage $ = _getTajirVestingWalletStorage();
    if ($.revoked) revert AlreadyRevoked();
    if (msg.sender != $.admin) revert NotAdmin();      // L122  admin-gated
    ...
    uint256 claimable = vested - alreadyReleased;      // L131
    if (currentBalance <= claimable) revert NothingToWithdraw();
    uint256 withdrawable = currentBalance - claimable;
    $.revoked = true;                                  // L137  global, irreversible
    ...
}
```

`_vestingSchedule` consults that same flag for **every** asset:

```
} else if ($.revoked || timestamp >= end()) {
    return totalAllocation;                            // L177-178  fully vested
}
```

Revoking against asset X therefore routes asset Y straight to fully-vested, regardless of elapsed time, cliff, or tranche count. There is no un-set path AlreadyRevoked at L121 makes it permanent.

The dangerous path requires no malice. An admin sweeping an unwanted or airdropped token out of the wallet is performing an ordinary operation. That one call fully vests the entire remaining allocation and permanently burns the one-shot clawback for every other asset.

A secondary path exists but requires role overlap. Because claimable and currentBalance are both read from the admin-supplied token, a token that merely reports a large balanceOf and no-ops its transfer satisfies the currentBalance > claimable guard at while moving nothing defeating the documented invariant that revoked cannot be set without moving a non-zero surplus. An honest admin has no incentive to do this, since it benefits the beneficiary at the protocol's expense; it becomes exploitable only where admin and beneficiary are the same key.

Impact

High

A single admin action irreversibly fully-vests every asset the wallet holds and permanently destroys the clawback right. The beneficiary can then release() 100% of the balance ahead of the intended cliff and tranche schedule. The economic guarantee of the vesting schedule is lost for the entire remaining term.

A related consequence was separately confirmed: because the revoked branch recomputes totalAllocation live, every post-revocation deposit is also 100% releasable on arrival.

Likelihood

Medium

Requires an admin transaction, but the triggering action sweeping an unwanted asset is routine and the destructive side effect is entirely unsignalled in the code.

POC

```
function test_revokeOnJunkTokenFullyVestsEverythingElse() public {
    // wallet holds the real allocation, still pre-cliff
    deal(address(tjr), address(wallet), 1_000_000e18);
    assertEq(wallet.vestedAmount(address(tjr), uint64(block.timestamp)), 0);

    // an unrelated junk token is airdropped in, admin sweeps it
    MockERC20 junk = new MockERC20();
    junk.mint(address(wallet), 1e18);

    vm.prank(admin);
    wallet.withdrawUnreleased(address(junk), payable(treasury));

```

```
12
13      // the entire TJR allocation is now fully vested, pre-cliff
14      assertEq(
15          wallet.vestedAmount(address(tjr), uint64(block.timestamp)),
16          1_000_000e18
17      );
18      // and the clawback is permanently gone
19      vm.prank(admin);
20      vm.expectRevert(TajirVestingWallet.AlreadyRevoked.selector);
21      wallet.withdrawUnreleased(address(tjr), payable(treasury));
22  }
```

Recommendation

Make revocation per-asset and assert that value actually moved.

```
- bool revoked;
+ mapping(address => bool) revoked;    // address(0) key == native
```

```
function _vestingSchedule(uint256 totalAllocation, uint64 timestamp)
    internal view override returns (uint256)
{
    // pass the asset being queried through, and gate on revoked[asset]
}
```

Because OpenZeppelin's _vestingSchedule signature does not carry the asset, this requires overriding vestedAmount(address,uint64) and vestedAmount(uint64) to consult the per-asset flag, or tracking the asset in transient storage for the duration of the call.

Additionally, assert a real balance delta so the flag cannot be flipped without value moving:

```
uint256 balBefore = native ? address(this).balance
                           : IERC20(token).balanceOf(address(this));
// ... perform transfer ...
uint256 balAfter  = native ? address(this).balance
                           : IERC20(token).balanceOf(address(this));
if (balBefore - balAfter == 0) revert NothingToWithdraw();
```

Team's Comment :

It is a requirement.
1. The contract is supposed to vest only one asset (most probably native in our case) per contract deployment but it is created in a way to tackle both native and ERC20 tokens.
2. Once revoked, this contract would be of no use and only remaining claims would be done.

So, it is intentional. No need for any update.

Medium Severity Issues

renounceOwnership() and transferOwnership() are inherited un-overridden

Acknowledged

Path

src/TajirVestingWallet.sol

Function Name

`renounceOwnership(), transferOwnership(address) (both inherited from OwnableUpgradeable)`

Description

TajirVestingWallet inherits OwnableUpgradeable through VestingWalletUpgradeable, where owner() is the vesting beneficiary every release() pays out to owner(). Neither renounceOwnership() nor transferOwnership() is overridden anywhere in the in-scope contracts.

The beneficiary can therefore call renounceOwnership(), setting owner() to address(0). Every subsequent release() then attempts to pay address(0): native transfers burn the value, and most ERC-20 implementations revert, permanently bricking the release path. The vesting wallet has no admin function to reassign the beneficiary, so this is unrecoverable.

transferOwnership() likewise allows the entire live vesting position including all future releases to be handed to an arbitrary address in a single unrestricted call, with no two-step accept and no admin oversight.

Impact

High. Permanent loss or misdirection of the entire vesting allocation.

Likelihood

Low. Requires the beneficiary to act against their own interest, or to be compromised. There is no attacker-controlled path.

POC

```
function test_beneficiaryCanStrandAllAssets() public {
    vm.prank(beneficiary);
    wallet.renounceOwnership();

    assertEq(wallet.owner(), address(0));

    vm.warp(block.timestamp + 400 days);      // fully vested
    vm.expectRevert();                        // ERC20 transfer to address(0)
    wallet.release(address(tjr));
}
```

Recommendation

Override both functions to revert, since neither has a legitimate use for a vesting wallet:

```
function renounceOwnership() public pure override {
    revert("TajirVestingWallet: beneficiary cannot be renounced");
}

```

```
function transferOwnership(address) public pure override {
    revert("TajirVestingWallet: beneficiary is immutable");
}
```

If beneficiary transfer is a required feature, implement it as a two-step propose/accept flow gated on admin approval rather than the inherited single-call form.

Medium Severity Issues

Unguarded subtraction underflows against fee on transfer tokens

Resolved

Path

src/TajirVestingWallet.sol

Function Name

`withdrawUnreleased(address,address payable), releasable(address) (inherited)`

Description

totalAllocation is recomputed live on every call as balanceOf(this) + released(token), while released(token) is a monotonically increasing accumulator. The subtraction is unguarded:

```
1  uint256 vested = native ? vestedAmount(uint64(block.timestamp))
2                          : vestedAmount(token, uint64(block.timestamp));
3  uint256 alreadyReleased = native ? released() : released(token);
4
5  uint256 claimable = vested - alreadyReleased;      // L131  unguarded
```

For a conserving ERC-20 the two move in lockstep and the difference is always safe the equivalent fuzz property against TajirToken passed, confirming the token itself is not the problem.

However, release(address token) and withdrawUnreleased(address token, ...) both accept an arbitrary caller-supplied token address. A rebasing token, a fee-on-transfer token, or any asset whose balance can decrease independently will drive balanceOf(this) below the already-accumulated released(token). vested then falls below alreadyReleased and the subtraction panics with 0x11 (arithmetic underflow).

The same expression exists in OpenZeppelin's inherited releasable(address), so the panic bricks both the clawback path and the beneficiary's normal release() path for that asset.

Impact

Medium. Permanent denial of service on the affected asset neither the beneficiary nor the admin can move it. Core protocol functionality is disabled for that token. Assets already correctly accounted for are not lost, but become unreachable.

Likelihood

Medium. Requires a non-conserving asset to enter the wallet. Since anyone can transfer any ERC-20 to the wallet address and the contract imposes no allowlist, this is externally triggerable by an unprivileged party

POC

```
function test_rebasingTokenBricksReleasePath() public {
    RebasingToken rt = new RebasingToken();
    rt.mint(address(wallet), 1000e18);

    vm.warp(block.timestamp + 400 days);        // fully vested
    wallet.release(address(rt));                // released(rt) accumulates

    rt.rebaseDown(50);                          // balance halves

    vm.expectRevert();                          // panic 0x11  underflow
    wallet.releasable(address(rt));
}
```

Recommendation:
Clamp the subtraction so a reduced balance degrades gracefully instead of reverting:

```
- uint256 claimable = vested - alreadyReleased;
+ uint256 claimable = vested > alreadyReleased ? vested - alreadyReleased : 0;
```

Additionally, restrict the accepted asset set to an explicit allowlist rather than any caller-supplied address. Balance-derived vesting accounting is only sound for conserving, non-rebasing, non-fee-on-transfer tokens; that assumption should be enforced in code rather than left implicit.

Medium Severity Issues

initialize() never validates startTimestamp

Acknowledged

Path

src/TajirVestingWallet.sol

Function Name

`initialize(...) (L82-111)`

Description

The initializer validates every schedule parameter except startTimestamp:

```
1  if (admin_ == address(0)) revert ZeroAddress();              // L92
2  if (upgrader_ == address(0)) revert ZeroAddress();           // L93
3  if (trancheDuration_ == 0) revert ZeroTrancheDuration();     // L94
4  if (totalTranches_ == 0) revert ZeroTotalTranches();         // L95
5  if ((totalTranches_ * trancheDuration_) + cliffSeconds != durationSeconds) {
6      revert InvalidScheduleParams();                          // L96-101
7  }
```

The coherence checkenforces that the schedule is arithmetically consistent, but never that it still locks anything. A startTimestamp sufficiently far in the past places cliff() and end() both behind block.timestamp, so _vestingSchedule returns totalAllocation on the very first call and the wallet is 100% vested at deployment.

Impact

Medium. A vesting wallet that enforces no lockup while presenting as a vesting contract. The cliff and tranche schedule are silently void; nothing on-chain distinguishes this from a correctly configured wallet.

Likelihood

Medium. Not reachable through the current deploy script, but reachable through the unprotected initializer through a future deployment path, or through operator error when back-dating a schedule.

POC

```
function test_pastStartTimestampVoidsLockup() public {
    uint64 pastStart = uint64(block.timestamp - 400 days);

    wallet.initialize(
        beneficiary, pastStart, 365 days, 30 days, 30 days, 11, admin, upgrader
    );

    deal(address(tjr), address(wallet), 1_000_000e18);

    // fully vested immediately  no cliff, no tranches
    assertEq(
        wallet.vestedAmount(address(tjr), uint64(block.timestamp)),
        1_000_000e18
    );
}
```

Recommendation

Require the schedule to lock value at the moment of initialization:

```
if (startTimestamp + cliffSeconds <= block.timestamp) {
    revert InvalidScheduleParams();
}
```

If back-dated schedules are an intended feature, bound how far into the past startTimestamp may be set and emit an event recording the deviation.

Team's Comment

Yes, the contract does not enforce a lower bound on startTimestamp but it is intended as the token allocations might be minted before the respective vesting contract is deployed. In those cases, the vesting start must be set to a past date to reflect the already-elapsed period. If we deployed an instance with a past date then it definitely is intentional.

Also, the admin role can always intervene if the schedule is deemed unsafe.

Medium Severity Issues

Clawback commits an irreversible flag with no balance-delta assertion

Acknowledged

Path

src/TajirVestingWallet.sol

Function Name

`withdrawUnreleased(address,address payable)`

Description

withdrawUnreleased derives the amount to sweep from two independent, uncached balanceOf reads against an unvalidated token contract, commits the irreversible global flag, emits an event asserting the amount, and only then performs the transfer:

```
1  uint256 currentBalance = native ? address(this).balance
2                                  : IERC20(token).balanceOf(address(this));   // L127
3  uint256 vested = ...;                                                       // L128
4  uint256 alreadyReleased = ...;                                              // L129
5  uint256 claimable = vested - alreadyReleased;                               // L131
6  if (currentBalance <= claimable) revert NothingToWithdraw();               // L133
7  uint256 withdrawable = currentBalance - claimable;                          // L135
8  $.revoked = true;                                                           // L137
9  emit VestingRevoked(token, withdrawable);                                   // L139  before transfer
10 if (native) Address.sendValue(to, withdrawable);                            // L141
11 else SafeERC20.safeTransfer(IERC20(token), to, withdrawable);               // L142
```

Note that vestedAmount(token, ...)performs a second balanceOf read internally. A token that returns different values across the two reads produces an inconsistent withdrawable.

Nothing reconciles the asserted amount against what was actually transferred. VestingRevoked is emitted with a value that may not match reality, which corrupts off-chain accounting and indexers that consume it.

Impact

Medium. Event data may not reflect the actual transfer, breaking downstream accounting. Combined with the missing delta assertion, the irreversible flag can be consumed while less value moves than reported.

Likelihood

Medium. Requires a non-standard or adversarial token to be passed by the admin.

POC

```
function test_eventAssertsUntransferredAmount() public {
    InconsistentToken it = new InconsistentToken();  // balanceOf varies per call
    it.mint(address(wallet), 1000e18);

    vm.expectEmit(true, false, false, false);
    emit VestingRevoked(address(it), 1000e18);       // asserted

    vm.prank(admin);
    wallet.withdrawUnreleased(address(it), payable(treasury));

    assertLt(it.balanceOf(treasury), 1000e18);       // actually received less
}
```

Recommendation

Cache the balance once, assert the realised delta, and emit the event after the transfer with the actual amount:

```
uint256 balBefore = _balanceOf(token);
// ... compute withdrawable from the cached value only ...
$.revoked = true;
if (native) Address.sendValue(to, withdrawable);
else SafeERC20.safeTransfer(IERC20(token), to, withdrawable);
uint256 moved = balBefore - _balanceOf(token);
if (moved == 0) revert NothingToWithdraw();
emit VestingRevoked(token, moved);
```

Team's Comment

The function is admin-only, we would do vesting only with trusted token contracts, and the logic guarantees that withdrawable is the exact amount that will be sent. So, the flag is intentionally irreversible because revoke is a one-time "emergency stop" that should not be undone.

Medium Severity Issues

Schedule parameters are read live with no clamp

Acknowledged

Path

src/TajirVestingWallet.sol

Function Name

`_vestingSchedule(uint256,uint64) (L170-183)`

Description

_vestingSchedule reads trancheDuration and totalTranches live from storage on every computation, with no clamp and no monotonicity guarantee:

```
1  uint256 tranchesPassed = (timestamp - cliff()) / $.trancheDuration;   // L180
2  return (totalAllocation * tranchesPassed) / $.totalTranches;          // L181
```

There is no setter for either value, so they cannot change during normal operation. However, the contract is UUPS upgradeable, and an upgrade can rewrite them directly in the ERC-7201 namespaced storage slot. Because vested amounts are recomputed from current values rather than snapshotted, changing either parameter retroactively alters how much is already vested including downward, below amounts already released.

Impact

Medium.An upgrade can silently rewrite the vesting schedule already in effect, changing the beneficiary's entitlement retroactively and potentially bricking releases.

Likelihood

Low. Requires an upgrade, which is gated on upgrader (the timelock in the intended configuration).

POC

```
1  function test_upgradeCanRetroactivelyShrinkVested() public {
2      vm.warp(block.timestamp + 200 days);
3      uint256 before = wallet.vestedAmount(address(tjr), uint64(block.timestamp));
4
5      // upgrade to an implementation that writes a larger totalTranches
6      vm.prank(upgrader);
7      wallet.upgradeToAndCall(address(new MaliciousVestingImpl()), "");
8
9      uint256 afterUpgrade = wallet.vestedAmount(address(tjr), uint64(block.timestamp));
10     assertLt(afterUpgrade, before);   // already-vested amount shrank
11 }
```

Recommendation
Snapshot the schedule at initialization and treat it as immutable thereafter, or enforce monotonicity so a recomputed vested amount can never fall below released()

```
uint256 computed = (totalAllocation * tranchesPassed) / $.totalTranches;
uint256 floor_ = released(token);
return computed < floor_ ? floor_ : computed;
```

Team's Comment
upgrader role would only be given to governance at initialization and making any change to the vesting schedule would require a governance-controlled upgrade, not in any other way. So, it's safe.

Low Severity Issues

Tranche quantization lets a revocation strip a nearly-elapsed tranche

Acknowledged

Path

src/TajirVestingWallet.sol

Function Name

`_vestingSchedule(uint256,uint64) withdrawUnreleased(address,address payable)`

Description

Vesting advances in discrete steps via floor division at L180 tranchesPassed = (timestamp - cliff()) / $.trancheDuration. Value earned in wall-clock time within an incomplete tranche is not credited until the tranche boundary is crossed.

Revocation timing is entirely unconstrained. An admin calling withdrawUnreleased at cliff + k·trancheDuration - 1 sweeps a full tranche of value that the beneficiary has, in wall-clock terms, almost entirely earned.

Impact

Low. The beneficiary loses up to one full tranche of value relative to a continuous-vesting interpretation. With the deployed configuration this is a bounded, single-tranche amount.

Likelihood

Medium. Requires deliberate timing by the admin. No technical barrier exists.

Recommendation

Consider pro-rata vesting within a tranche, or introduce a notice period so revocation takes effect only at the next tranche boundary. If discrete tranches are an intentional design decision, document the timing exposure explicitly for beneficiaries.

Team's Comment :

This tranches schedule is an intentional design decision.

Low Severity Issues

TajirToken.initialize silently rescales initialSupply

Acknowledged

Path

src/token/TajirToken.sol

Function Name

`initialize(string,string,uint256,address,address) (L37-56)`

Description

The zero-check at L46 validates the raw argument, while L55 mints a value scaled by 18 decimals:

```
1  if (initialSupply == 0) revert Tajir__ZeroSupply();     // L46  checks raw value
2  ...
3  _mint(initialHolder, initialSupply * 10 ** decimals());
```

The parameter is documented as "whole-token initial supply (decimals applied internally)" at L34, so the behaviour is intentional. However, a caller who passes an already-scaled value a very common convention, silently deploys a supply 10^18 times larger than intended. There is no upper bound to catch this.

Impact

Low. A misconfigured genesis supply. The error is severe if it occurs, but immediately visible on-chain post-deployment and correctable only by redeployment.

Likelihood

Low. Requires deployment-time operator error; the NatSpec does document the expected unit.

Recommendation

Add a sanity bound on the raw argument, for example if (initialSupply > 1e12) revert Tajir__ZeroSupply();, or accept the pre-scaled value directly and rename the parameter to initialSupplyWei to eliminate the ambiguity.

Team's Comment :

Genesis supply won't be misconfigured as Admins would be doing this and it is a one-time mint at initialization.

Informational Severity Issues

revoked has no getter and its guard precedes the authorization gate

Resolved

Path

src/TajirVestingWallet.sol

Function Name

`withdrawUnreleased(address,address payable)`

Description

trancheDuration(), totalTranches(), and upgrader() all have public getters (L148-164), but revoked the single most consequential state variable in the contract, and terminal once set has none. Off-chain consumers cannot determine whether a wallet has been revoked without replaying VestingRevoked events.

Separately, the ordering at L121-122 checks AlreadyRevoked before NotAdmin, so an unauthorized caller can distinguish revoked from non-revoked wallets by the revert reason.

Impact

No security impact. Observability and information-disclosure hygiene.

Recommendation

Add function revoked() external view returns (bool) and swap the two checks so authorization is evaluated first.

Informational Severity Issues

withdrawUnreleased re-derives releasable() inline

Resolved

Path

src/TajirVestingWallet.sol

Function Name

`withdrawUnreleased(address,address payable) (L127-131)`

Description

L127-131 recomputes vested - alreadyReleased inline rather than calling the inherited releasable(token) helper, which computes exactly the same quantity. Two definitions of one concept now exist in the codebase and can drift apart under future edits a maintenance hazard that also doubles the number of balanceOf calls to an untrusted token.

Impact

No security impact today. Maintainability and gas.

Recommendation

Call the inherited releasable(token) / releasable() helper and cache the result.

Informational Severity Issues

Floor division routes rounding remainders away from the beneficiary

Resolved

Path

src/TajirVestingWallet.sol

Function Name

`_vestingSchedule(uint256,uint64)`

Description

(totalAllocation * tranchesPassed) / $.totalTranches truncates on every computation, so the remainder consistently favours the contract over the beneficiary until end() is reached, at which point the timestamp >= end() branch returns the full allocation and settles the difference. Any balance smaller than totalTranches vests as zero for the entire schedule.

Impact

No loss at maturity. Dust-scale under-crediting mid-schedule.

Recommendation

Acceptable as-is given the terminal branch settles exactly. Document the rounding direction.

Informational Severity Issues

upgradeToAndCall inherited payable with no ETH-handling path

Acknowledged

Path

src/TajirVestingWallet.sol

Function Name

`upgradeToAndCall(address,bytes) (inherited from UUPSUpgradeable)`

Description

TajirToken inherits UUPSUpgradeable.upgradeToAndCall, which is payable, but declares no receive(), no fallback(), and no sweep function. Any ETH sent with an upgrade call is permanently locked in the contract.

Impact

Unrecoverable loss of any ETH sent during an upgrade. Zero under correct usage.

Recommendation

Override upgradeToAndCall to reject non-zero msg.value, or add an admin-gated ETH sweep.

Team's Comment

As the contract never needs to receive native ether and upgrade workflow is strictly controlled by a timelock that always sends msg.value = 0, so no ether can be unintentionally locked.

Functional Tests

Initialization

- ✓ Should initialize with valid parameters and set beneficiary, admin, upgrader, trancheDuration and totalTranches correctly
- ✓ Should revert when admin_ is the zero address
- ✓ Should revert when upgrader_ is the zero address
- ✓ Should revert when trancheDuration_ is zero
- ✓ Should revert when totalTranches_ is zero
- ✓ Should revert when (totalTranches * trancheDuration) + cliffSeconds != durationSeconds
- ✓ Should revert on a second call to the 8-argument initialize
- ✓ Should revert on the inherited 3-argument initialize after the 8-argument path has run
- ✓ Should not allow the inherited 3-argument initialize to be called on an uninitialized proxy
- ✓ Should revert when startTimestamp places the cliff in the past
- ✓ Should revert when admin, upgrader and beneficiary are not distinct
- ✓ Should confirm the implementation contract itself is permanently disabled by _disableInitializers()

Vesting Schedule

- ✓ Should return zero vested before cliff() for both native and ERC-20 tracks
- ✓ Should return zero vested at exactly cliff() - 1
- ✓ Should vest exactly one tranche at cliff() + trancheDuration
- ✓ Should vest exactly k tranches at cliff() + k·trancheDuration
- ✓ Should not advance vesting between tranche boundaries
- ✓ Should return the full allocation at end()

- ✓ Should return the full allocation for any timestamp beyond end()
- ✓ Should handle totalTranches == 1 correctly
- ✓ Should return zero for a balance smaller than totalTranches until end()

Release

- ✓ Should release the vested amount to the beneficiary for ERC-20
- ✓ Should release the vested amount to the beneficiary for native
- ✓ Should be callable by any address while still paying owner()
- ✓ Should release nothing when called twice in the same tranche window
- ✓ Should accumulate released(token) monotonically across releases
- ✓ Should revert when owner() is the zero address after renounceOwnership()
- ✓ Should not revert for a conserving token at any point in the schedule

Clawback / revocation

- ✓ Should allow admin to withdraw the unvested surplus
- ✓ Should revert with NotAdmin for any non-admin caller
- ✓ Should revert with ZeroAddress when to is the zero address
- ✓ Should revert when to == address(this)
- ✓ Should revert with NothingToWithdraw when currentBalance <= claimable
- ✓ Should revert with AlreadyRevoked on a second call
- ✓ Should leave the claimable (vested but unreleased) portion in the contract
- ✓ Should not fully vest other assets when one asset is revoked
- ✓ Should not allow the flag to be set without a non-zero balance delta
- ✓ Should emit VestingRevoked with the amount actually transferred
- ✓ Should handle native revocation via address(0) correctly

Access Control And Upgradeability

- ✔ Should allow only upgrader to authorize an upgrade on the vesting wallet
- ✔ Should revert with NotUpgrader for any other caller
- ✔ Should preserve ERC-7201 namespaced storage across an upgrade
- ✔ Should not allow an upgrade to reduce vested below released()
- ✔ Should allow only UPGRADER_ROLE to upgrade TajirToken
- ✔ Should prevent renouncing the last DEFAULT_ADMIN_ROLE holder

Tajirtoken

- ✔ Should mint exactly initialSupply * 10 ** decimals() to initialHolder
- ✔ Should revert when initialHolder is the zero address
- ✔ Should revert when upgrader is the zero address
- ✔ Should revert when initialSupply is zero
- ✔ Should have no mint or burn surface after initialization
- ✔ Should support EIP-2612 permit with a valid signature
- ✔ Should reject a replayed permit signature
- ✔ Should reject a permit signature past its deadline
- ✔ Should produce a domain separator bound to the correct chainId

Non-Standard Token Handling

- ✔ Should not brick releasable() when a rebasing token's balance decreases
- ✔ Should not brick releasable() for a fee-on-transfer token
- ✔ Should behave correctly for a token returning no boolean on transfer
- ✔ Should behave correctly for a token that reverts on zero-value transfer

Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
VestingWalletUpgradeable (OpenZeppelin)	Base Contract	Provides released/releasable/release, the vestedAmount accumulators, and owner() as beneficiary	Correct linear-vesting accounting; initializer semantics as documented	Exposes a second public initializer that is not overridden; maintains two independent accumulators
VestingWalletCliffUpgradeable (OpenZeppelin)	Base Contract	Provides cliff() and the pre-cliff zero branch	Cliff is enforced before any other schedule logic	Cliff ordering relative to the revoked branch was verified correct by fuzzing
OwnableUpgradeable (OpenZeppelin)	Base Contract	owner() is the vesting beneficiary; payout target for every release	Owner is the intended beneficiary and acts rationally	renounceOwnership/ transferOwnership are not overridden beneficiary can strand or transfer the position

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
UUPSUpgradeable (OpenZeppelin)	Base Contract	Upgrade mechanism for both the vesting wallet and TajirToken	Owner is trusted and competent; new owner explicitly accepts ownership	Upgrade can rewrite schedule parameters retroactively inherited upgradeToAndCall is payable with no ETH path
AccessControlUpgradeable (OpenZeppelin)	Base Contract	Role management for TajirToken (DEFAULT_ADMIN_ROLE, UPGRADER_ROLE)	Recovers signer correctly; rejects malleable signatures (OZ v5 behavior)	Both roles granted to one key with no timelock renounceRole can permanently freeze upgrades
ERC1967Proxy (OpenZeppelin)	Base Contract	Proxy implementation for TajirTokenProxy and the vesting wallets	Standard ERC-1967 slot handling and atomic init when data is supplied	Deployed with empty init data, creating the front-running window of; wrapper drops payable
ERC20PermitUpgradeable (OpenZeppelin)	Base Contract	EIP-2612 gasless approvals for TajirToken	Correct EIP-712 domain construction, nonce handling, and ecrecover usage	Standard OpenZeppelin implementation; no deviation found in this audit
SafeERC20 (OpenZeppelin)	Library	Wraps ERC-20 transfers in the clawback path (safeTransfer)	Correctly handles non-standard return values and reverts on failure	Does not protect against fee-on-transfer or rebasing accounting drift

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Address (OpenZeppelin)	Library	sendValue for the native clawback path	Forwards all gas and bubbles reverts	A contract recipient with a reverting receive() blocks the native clawback
Caller-supplied token address	External ERC-20 Contract	Passed directly into release(token) and withdrawUnreleased(token, to); its balanceOf drives all accounting	Unvalidated. Assumed conserving, non-rebasing, non-fee-on-transfer, honest balanceOf and transfer	A lying balanceOf/no-op transfer flips the revoke flag at zero cost (); a balance-reducing token bricks releases
Tajir Chain (chainId 7733)	Blockchain / Execution Environment	Deployment target for vesting wallets; TJR is the native gas asset here	Native asset is TJR; no ERC-20 TJR representation exists on this chain	If a wrapped WTJR is introduced, the dual-accumulator path becomes exploitable

2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets one table.

Contract Name	Function	Category
TajirVestingWallet.sol	constructor()	**What this function can do** Disable initializers on the implementation contract **What this function cannot/should not do** Initialize any proxy state; protect the proxy from being initialized by a third party **Main invariant(s)** The implementation contract can never be initialized directly **Access level** Implicit (once at deployment)
TajirVestingWallet.sol	initialize(address,uint64,uint64,uint64,uint64,uint64,address,address)	**What this function can do** Set beneficiary (as owner()), start, duration, cliff, tranche schedule, admin and upgrader, exactly once **What this function cannot/should not do** Be callable twice; be callable by an unintended party before the deployer; accept a schedule that locks nothing; permit admin/upgrader/beneficiary to be the same key **Main invariant(s)** (totalTranches × trancheDuration) + cliffSeconds == durationSeconds; runs exactly once

Contract Name	Function	Category
TajirVestingWallet.sol	initialize(address,uint64,uint64) (inherited, not overridden)	**Access level** Permissionless anyone may call it on an uninitialized proxy **What this function can do** Set owner(), start and duration, consuming the one-shot initializer flag **What this function cannot/should not do** Exist at all on this contract it bypasses every schedule validation and leaves admin, upgrader, trancheDuration and totalTranches at zero **Main invariant(s)** None enforced
TajirVestingWallet.sol	withdrawUnreleased(address,address payable)	**Access level** Permissionless, unintended entry point **What this function can do** Sweep the unvested surplus of one caller-chosen asset to a caller-chosen recipient, once, and latch the revoke flag **What this function cannot/should not do** Touch the claimable (vested-but-unreleased) portion; affect the vesting schedule of other assets; latch the flag without moving value; send to address(this); be called twice **Main invariant(s)** Claimable portion remains in the contract; the flag is one-shot and terminal **Access level** admin only (NotAdmin, L122)

Contract Name	Function	Category
TajirVestingWallet.sol	release() / release(address) (inherited)	**What this function can do** Transfer the vested-but-unreleased amount of native or an ERC-20 to owner() **What this function cannot/should not do** Pay any address other than owner(); release more than vested; revert for a conserving token **Main invariant(s)** released ⩽ vested; released is monotonically non-decreasing **Access level** Permissionless (funds always route to owner())
TajirVestingWallet.sol	renounceOwnership() / transferOwnership(address) (inherited)	**What this function can do** Set the beneficiary to the zero address, or reassign it to an arbitrary address, in one unrestricted call **What this function cannot/should not do** Be reachable at all on a vesting wallet the beneficiary should be fixed for the schedule's lifetime **Main invariant(s)** Violated: assets should always have a valid recipient **Access level** owner() (the beneficiary) unintended capability

Contract Name	Function	Category
TajirVestingWallet.sol	upgradeToAndCall(address, bytes) (inherited UUPS)	**What this function can do** Replace the implementation, including logic governing the schedule **What this function cannot/should not do** Alter already-vested entitlements retroactively; break ERC-7201 storage layout **Main invariant(s)** Vested amounts should never decrease below released() **Access level** upgrader via _authorizeUpgrade (NotUpgrader)
TajirVestingWallet.sol	trancheDuration() / totalTranches() / upgrader()	**What this function can do** Expose schedule and role configuration for off-chain verification **What this function cannot/should not do** Mutate state **Main invariant(s)** N/A (view) **Access level** Public view. Note: no equivalent getter exists for revoked

Contract Name	Function	Category
TajirVestingWallet.sol	receive() (inherited)	**What this function can do** Accept native asset into the wallet, increasing the native track's totalAllocation **What this function cannot/should not do** Accept value in a way that inflates an already-revoked allocation **Main invariant(s)** Post-revocation deposits should not be instantly 100% releasable **Access level** Permissionless
TajirToken.sol	constructor()	**What this function can do** Disable initializers on the implementation **What this function cannot/should not do** Initialize proxy state **Main invariant(s)** Implementation is never directly initializable **Access level** Implicit (once at deployment)
TajirToken.sol	initialize(string,string,uint256,address,address)	**What this function can do** Set name/symbol, mint the full fixed supply to initialHolder, grant DEFAULT_ADMIN_ROLE and UPGRADER_ROLE **What this function cannot/should not do** Run twice; mint again afterwards; grant both roles to a single key without a timelock

Contract Name	Function	Category
TajirToken.sol	_authorizeUpgrade(address)	**Main invariant(s)** Total supply is fixed at initialization; no mint/burn surface exists thereafter **Access level** Permissionless on an uninitialized proxy the same atomic-deployment concern as Issue 1 applies **What this function can do** Gate implementation replacement on UPGRADER_ROLE **What this function cannot/should not do** Permit upgrade by a key that also administers its own role without delay **Main invariant(s)** Only UPGRADER_ROLE may upgrade **Access level** UPGRADER_ROLE (currently the same key as DEFAULT_ADMIN_ROLE)
TajirToken.sol	permit(...) / nonces(address) / DOMAIN_SEPARATOR() (inherited)	**What this function can do** Approve by EIP-712 signature without an on-chain transaction from the owner **What this function cannot/should not do** Permit signature replay, cross-chain replay, or use past the deadline **Main invariant(s)** Each nonce is consumed exactly once; domain separator binds to chainId **Access level** Permissionless with a valid signature

Contract Name	Function	Category
TajirToken.sol	transfer / transferFrom / approve (inherited ERC20)	**Main invariant(s)** Total supply is fixed at initialization; no mint/burn surface exists thereafter **Access level** Permissionless on an uninitialized proxy the same atomic-deployment concern as Issue 1 applies **What this function can do** Standard ERC-20 value movement and allowance management **What this function cannot/should not do** Deviate from ERC-20 semantics; apply fees or rebase **Main invariant(s)** Sum of balances equals total supply; supply is constant **Access level** Permissionless
TajirToken.sol	constructor(address,bytes)	**What this function can do** Deploy an ERC-1967 proxy pointing at an implementation, optionally executing init calldata atomically **What this function cannot/should not do** Be deployed with empty data when the implementation has an unprotected initializer **Main invariant(s)** mplementation slot set exactly once at construction **Access level** Implicit (once at deployment). Note: not payable, unlike the base

3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets
TajirVestingWallet.sol	Native (TJR on chainId 7733)	Vesting allocation held on the native track
TajirVestingWallet.sol	ERC20	Any caller-supplied token transferred to the wallet, including the L1 TajirToken
TajirTokenProxy.sol	ERC20	Holds no external value; custodies the full TajirToken supply ledger in its own storage

3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
TajirVesting Wallet.sol	receive()	Native	–	Public	Post-revocation deposits are instantly 100% releasable
TajirVesting Wallet.sol	(direct ERC20 transfer in)	ERC20	–	Public	No allowlist a rebasing or fee-on-transfer token bricks the release path ; an airdropped junk token invites the revoke footgun
TajirVesting Wallet.sol	release()	–	Native	Public	Underflow panic against balance-reducing assets
TajirVesting Wallet.sol	withdrawUnreleased(address, address payable)	–	Native or ERC20	admin	One-shot and irreversible; sweeps the unvested surplus, fully vests all other assets , no balance-delta assertion , to == address(this) permitted
TajirVesting Wallet.sol	upgradeToAndCall(address, bytes)	–	–	upgrader	No direct value movement, but can rewrite schedule logic governing every future exit

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
TajirToken.sol	initialize(...)	–	ERC20 (minted)	Public on uninitialized proxy	Mints the entire fixed supply to initialHolder; supply is scaled by 1e18 internally
TajirToken.sol	transfer / transferFrom	ERC20	ERC20	Public	Standard ERC-20; no fee, no rebase
TajirToken.sol	upgradeToAndCall(address, bytes)	ETH (locked)	-	UPGRADER_ROLE	Inherited payable with no ETH-handling path value sent is unrecoverable

Fuzzing Results

Invariants Tested	Status (Passed or Fail)
releasableNeverReverts releasable() never reverts for a conserving asset at any point in the schedule	Passed
releasedNeverExceedsVested released(token) ⩽ vestedAmount(token, now) at all times	Passed
solvent contract balance always covers the unreleased vested portion	Passed
preCliffNothingReleasable no value is releasable at any timestamp before cliff()	Passed
releasableNeverReverts - releasable() never reverts for a conserving asset at any point in the schedule	Passed
releasedNeverExceedsVested - released(token) never exceeds vestedAmount(token, now)	Passed
solvent - contract balance always covers the unreleased vested portion	Passed
preCliffNothingReleasable - no value is releasable at any timestamp before cliff()	Passed
vestedBoundedByAllocation - vested amount never exceeds total allocation for any asset or timestamp	Passed
tokenConservation - assets in equals assets out plus balance held, across all paths	Passed

Invariants Tested	Status (Passed or Fail)
beneficiaryReceiptsMatchAccumulator - value received by owner() equals released()	Passed
initializedWalletHasValidatedParams - an initialized wallet always has non-zero trancheDuration and totalTranches	Failed 1-call counterexample: a proxy initialized through the inherited 3-argument initializer bypasses every parameter check, leaving both schedule values at zero
ownerIsIntendedBeneficiary - owner() equals the beneficiary supplied by the deployer	Failed -call counterexample: any caller may initialize an unclaimed proxy and install themselves as the payout recipient
tjrScheduleUnaffectedByOtherAssetRevoke - revoking asset X does not alter the vesting schedule of asset Y	Failed 6-call counterexample: a single global revocation flag routes every unrevoked asset to fully-vested regardless of elapsed time
clawbackSurvivesForUnrevokedAsset - the clawback right for asset Y survives a revocation against asset X	Failed - 6-call counterexample: the one-shot flag is consumed for all assets simultaneously, permanently destroying the clawback right
revokeRequiresNonZeroSurplusMoved - the revocation flag cannot be latched without a non-zero balance delta	Failed - counterexample using a token that reports a large balanceOf and no-ops its transfer, satisfying the surplus guard while moving nothing
heldAssetsAlwaysHaveARecipient - every asset held always has a non-zero payout recipient	Failed - counterexample using a token that reports a large balanceOf and no-ops its transfer, satisfying the surplus guard while moving nothing
vestedNeverBelowReleased - vested amount never falls below the monotonic released accumulator	Failed - 8-call counterexample: an asset whose balance decreases independently of any release drives the recomputed allocation below the accumulator, panicking the subtraction. Reproduced deterministically; confirmed confined to the affected asset, with conserving assets in the same wallet unaffected

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Closing Summary

In this report, we have considered the security of Tajir Chain. We performed our audit according to the procedure described above.

13 Issues in Tajir Chain.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With seven years of expertise, we've secured over 1400 projects globally, averting over $3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

8+ Years of Expertise	**1M+** Lines of Code Audited
50+ Chains Supported	**1500+** Projects Secured

Follow Our Journey

AUDIT REPORT

July 2026

For

TAJIR CHAIN

QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com